

A Project Report

On

CAFETERIA MANAGEMENT AND ORDERING SYSTEM

Submitted in partial fulfilment of requirements for the award of the course

of

CGB1122 – DATA STRUCTURES

Under the guidance of

Mrs. R. K. DEEBIKA M.E.,

Assistant Professor/AI&DS

Submitted by

VINDHIYAA P (927625BCS187)

ANUSHKA M (927625BSC003)

SWEDHACHELLAM K (927625BSC058)

DEPARTMENT OF FRESHMEN ENGINEERING

M.KUMARASAMY COLLEGE OF ENGINEERING
(Autonomous)

KARUR – 639 113

JUNE 2026

M. KUMARASAMY COLLEGE OF ENGINEERING
(Autonomous Institution affiliated to Anna University, Chennai)
KARUR – 639 113

BONAFIDE CERTIFICATE

Certified that this project report on “**CAFETERIA MANAGEMENT AND ORDERING SYSTEM**” is the bonafide work of **VINDHIYAA P (927625BCS187)**, **ANUSHKA M (927625BSC003)**, **SWEDHACHELLAM K (927625BSC058)**, who carried out the project work during the academic year 2025 - 2026 under my supervision.

Signature

Mrs. R. K. DEEBIKA M.E.,
SUPERVISOR,

Department of Artificial Intelligence
and Data Science,
M. Kumarasamy College of Engineering,
Thalavapalayam, Karur -639 113.

Signature

Dr. K. CHITIRAKALA M.Sc.,M.Phil.,Ph.D.,
HEAD OF THE DEPARTMENT,

Department of Freshmen Engineering,
M. Kumarasamy College of Engineering,
Thalavapalayam, Karur -639 113.

Submitted for the End Semester Review held on _____

INTERNAL EXAMINER

VISION AND MISSION OF THE INSTITUTION

VISION OF THE INSTITUTION

To emerge as a leader among the top institutions in the field of technical education

MISSION OF THE INSTITUTION

- Produce smart technocrats with empirical knowledge who can surmount the global challenges
- Create a diverse, fully-engaged, learner-centric campus environment to provide quality education to the students
- Maintain mutually beneficial partnerships with our alumni, industry, and Professional associations

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

PO1: Engineering knowledge: Apply knowledge of mathematics, natural science computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature and analyse complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/development of solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyse and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

ABSTRACT

The Cafeteria Management and Ordering System is developed using the C programming language by applying fundamental data structures such as arrays, queues, and stacks to create an efficient and user-friendly ordering system. In this project, arrays are used to store and manage the menu items, enabling quick access to food names, prices, and categories. The queue data structure is implemented to handle customer orders in a first-in-first-out (FIFO) manner, ensuring that each order is processed fairly based on the order of arrival. Multiple food items can be added to the queue as part of a single or multiple customer orders. Additionally, a stack is used to provide an undo feature, allowing users to remove the most recently added item in case of mistakes during ordering. The system also includes an automatic billing feature that calculates the total amount based on selected items and their quantities. It further incorporates discount and tax calculations to generate an accurate final bill. This project highlights how data structures can be effectively used to build practical applications that improve efficiency, reduce errors, and simplify cafeteria management operations.

ABSTRACT WITH POs MAPPING

ABSTRACT	POs MAPPED
The Cafeteria Management and Ordering System is developed using the C programming language with data structures like arrays, queues, and stacks to make the ordering process efficient and simple. Arrays are used to store and manage menu items such as food names and prices, allowing quick access. The queue is used to handle customer orders in a first-in-first-out (FIFO) manner, ensuring fair processing. A stack is implemented to provide an undo feature, enabling users to remove the last added item easily. The system also includes automatic bill calculation based on selected items, along with discount and tax computation. Overall, this project demonstrates the practical use of data structures in building an effective cafeteria management system.	PO1(2) PO2(2) PO3(3) PO5(3) PO8(3) PO9(2)

Note: 1- Low, 2-Medium, 3- High

SUPERVISOR

HEAD OF THE DEPARTMENT

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
1	Introduction	1
	1.1 Introduction	1
	1.2 Objective	1
	1.3 Data Structure Choice	1
2	Project Methodology	2
	2.1 Methodologies	2
	2.2 Block Diagram	4
3	Modules	5
	3.1 Menu Module	5
	3.2 Cart Module	5
	3.3 Undo Module (Stack)	6
	3.4 Order Processing Module (Queue)	6
	3.5 Billing Module	6
4	Results and Discussion	7
	4.1 Result	7
	4.2 Discussion	11
5	Conclusion	12
	References	13
	Appendix	14

CHAPTER 1

INTRODUCTION

1.1 Introduction

The Cafeteria Management and Ordering System is a simple and efficient application developed using the C programming language to manage food ordering in a cafeteria. In many cafeterias, the ordering process is often done manually, which can lead to delays, errors, and confusion in handling multiple customer requests. This project aims to overcome such issues by automating the process using fundamental data structures. Arrays are used to store and display menu items, queues are used to manage customer orders in a systematic first-in-first-out manner, and stacks are used to provide an undo feature for correcting mistakes. The system also includes automatic bill calculation with discount and tax, ensuring accuracy and saving time. Overall, this project demonstrates how basic data structures can be applied to solve real-world problems and improve efficiency in cafeteria management.

1.2 Objective

The objective of this project is to develop a Cafeteria Management and Ordering System using C programming and data structures. It aims to efficiently manage menu items using arrays and handle customer orders using a queue for fair processing. The project also focuses on implementing a stack to provide an undo feature for order correction. Additionally, it ensures accurate and automatic bill calculation including discount and tax. Overall, the objective is to simplify and improve cafeteria operations.

1.3 Data Structure Choice

The choice of data structures in the Cafeteria Management and Ordering System is based on the need for efficient data handling and smooth operation. **Arrays** are used to store and manage menu items because they allow easy access and fast retrieval of food details like name and price. A **queue** is chosen to manage customer orders as it follows the **First-In-First-Out (FIFO)** principle, ensuring fair processing. A **stack** is used to implement the undo feature, allowing removal of the most recently added item in case of mistakes as it follows the **Last-In-First-Out (LIFO)** principle. These data structures are simple, effective, and well suited for handling cafeteria operations in an organized manner.

CHAPTER 2

PROJECT METHODOLOGY

2.1 Methodologies

The Cafeteria Management and Ordering System is designed using basic data structures such as arrays, **Queue** uses **First-In-First-Out (FIFO)** principle and **Stack** that follows **Last-In-First-Out (LIFO)** and it has only one pointer named as **Top** pointer to manage menu items, customer orders, and order modifications efficiently. Each data structure is selected based on its functionality and suitability for a specific task in the system. These structures help in organizing data, improving performance, and simplifying operations within the application.

1. Array Methodology

Arrays are used to store and manage cafeteria menu items such as food names, item IDs, prices, and categories in an organized manner. This methodology allows quick access, easy retrieval, and efficient display of menu information to users. Arrays help in maintaining the menu systematically and simplify data handling operations within the application.

2. Queue Methodology

The queue data structure is implemented to manage customer orders efficiently using the First-In-First-Out (FIFO) principle. Orders are processed in the same sequence in which they are placed, ensuring fairness and smooth order management. This methodology helps avoid confusion during peak hours and improves the efficiency of cafeteria operations.

3. Stack Methodology

The stack data structure is used to implement the undo operation in the ordering process. It follows the Last-In-First-Out (LIFO) principle, where the most recently added item can be removed easily in case of ordering mistakes. This methodology improves user convenience and provides flexibility during order placement.

4. Automated Billing System

The automated billing methodology is used to generate accurate bills instantly based on selected food items and quantities. The system automatically calculates subtotal, discount, tax, and final payable amount without manual calculation. This reduces billing errors, saves time, and increases transaction reliability within the cafeteria management system.

5. Efficient Data Handling

Efficient data handling techniques are used throughout the system to ensure smooth storage, retrieval, updating, and processing of customer and order information. Proper management of data structures improves system performance, reduces processing time, and enhances the overall reliability of the application. This methodology ensures that cafeteria operations are carried out in a fast, organized, and error-free manner.

Key Features of Data Structures

The Cafeteria Management and Ordering System uses different data structures to improve the performance and efficiency of the application. **Arrays** provide simple and fast storage of menu items, allowing easy access and retrieval of food details. The **Queue** data structure ensures fair processing of customer orders by following the **First-In-First-Out (FIFO) principle**. The **Stack** data structure supports undo operations by following **the Last-In-First-Out (LIFO) principle**, helping users correct recent actions easily. These data structures improve data organization, reduce processing time, simplify order management, and enhance the overall reliability and performance of the cafeteria management system.

2.2 Block Diagram

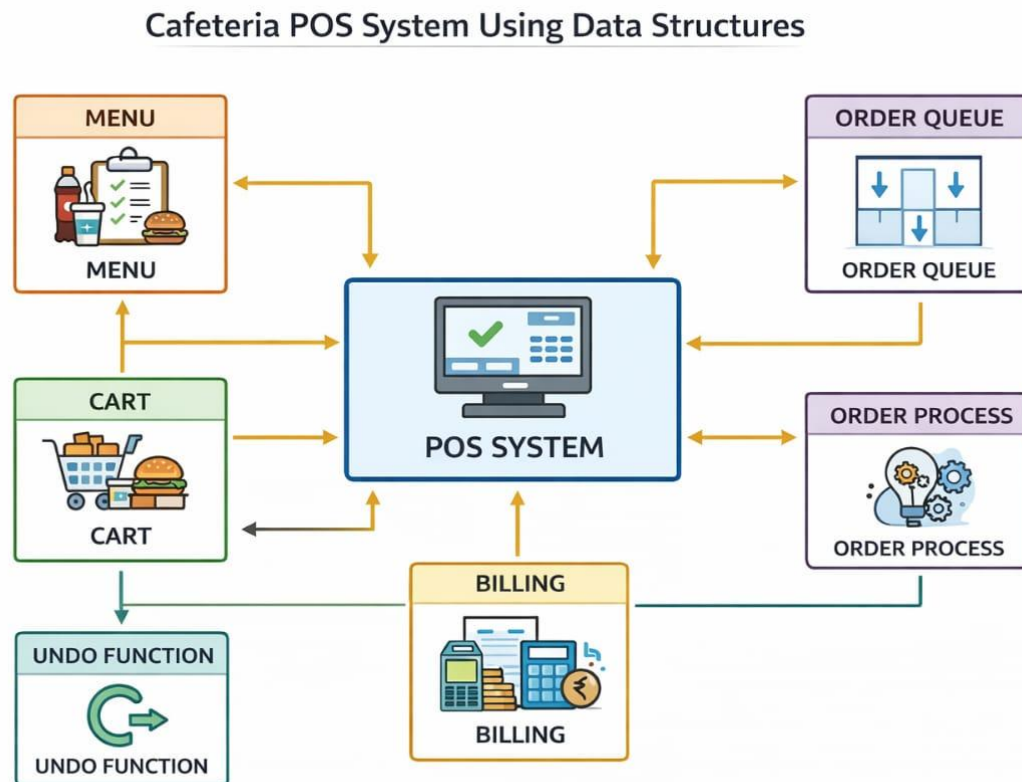


Fig 2.1 CAFETERIA MANAGEMENT AND ORDERING SYSTEM BLOCK DIAGRAM

Figure 2.1 shows the complete flow of the Cafeteria Management and Ordering System. The system starts with the menu module where food items are stored and displayed using Arrays. Customers can select food items and add them to the cart, while stack operations are used to provide undo functionality for correcting recent actions. After confirming the order, the queue module processes customer orders using the First-In-First-Out (FIFO) principle to ensure fair order handling. Finally, the billing module automatically calculates the total amount, including discount and tax, and generates the final bill.

CHAPTER 3

MODULES

3.1 Menu Module

The Menu Module is one of the core components of the Cafeteria Management and Ordering System. This module is responsible for storing, organizing, and displaying all available food items in the cafeteria. Arrays are used to maintain menu details such as item ID, food name, category, and price in a structured format. Customers can easily view the available menu before placing an order. The module provides quick access to menu information and allows smooth navigation through different food items. It also helps the administrator update or manage menu details whenever required. Proper organization of menu data improves system efficiency and reduces confusion during ordering. This module acts as the foundation of the entire cafeteria ordering process.

3.2 Cart Module

The Cart Module is used to temporarily store the food items selected by customers before confirming the order. When users choose food items from the menu, the selected items along with their quantities are added to the cart. This module allows customers to review their order details, modify quantities, or add more items before final submission. It improves user convenience by providing flexibility during the ordering process. The cart module also keeps track of the total number of selected items and their prices. By maintaining temporary order information, this module helps avoid ordering mistakes and ensures a smooth transition between menu selection and final order placement.

3.3 Undo Module (Stack)

The stack follows the Last-In-First-Out (LIFO) principle, where the most recently added item can be removed first. Whenever a customer adds an item to the cart, the details are pushed into the stack. If the user makes a mistake or wants to cancel the latest action, the undo operation removes the last added item from both the cart and stack. This feature improves user experience by allowing quick corrections without restarting the entire ordering process. The module reduces user errors and makes the application more interactive and user-friendly.

3.4 Order Module And Process Module (Queue)

The Order Processing Module manages customer orders using a queue data structure. The queue follows the First-In-First-Out (FIFO) principle, ensuring that orders are processed in the exact order in which they are received. Once customers confirm their orders, the order details are added to the queue for processing. This methodology helps maintain fairness and avoids confusion when handling multiple customer requests simultaneously. The queue module ensures smooth order flow inside the cafeteria and supports efficient management during busy hours.

3.5 Billing Module

The Billing Module is responsible for generating the final bill automatically after order confirmation. It calculates the total amount based on selected food items and their quantities.. Automated billing reduces manual calculation errors and saves time during transactions. The generated bill clearly displays item names, quantities, individual prices, subtotal, discount amount, tax amount, and final payable amount. This module improves transaction reliability and provides a smooth payment experience for both customers and cafeteria staffs.

CHAPTER 4

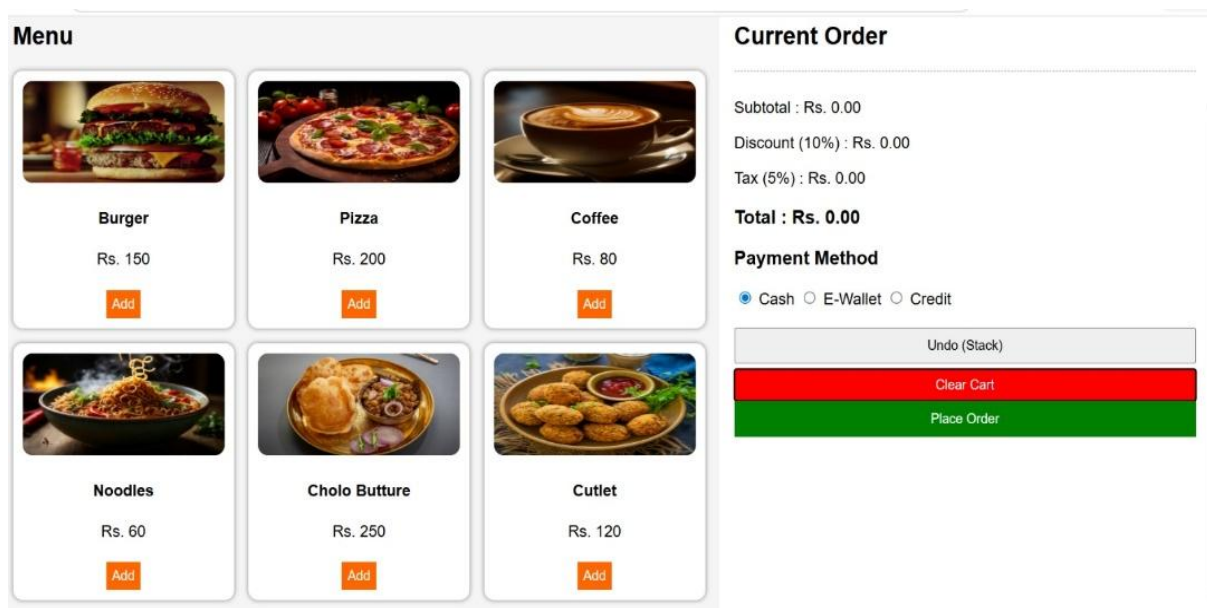
RESULTS AND DISCUSSION

4.1 Results

Main Menu Interference

```
1.Menu  2.Add  3.Undo  4.Bill  5.Place Order  6.Process  7.Exit
Enter choice: 1

----- MENU -----
1. Burger - Rs 150.00
2. Pizza - Rs 200.00
3. Coffee - Rs 80.00
4. Noodles - Rs 60.00
5. Cholo Bhature - Rs 250.00
6. Cutlet - Rs 120.00
```



Adding Items To Cart

```
1.Menu 2.Add 3.Undo 4.Bill 5.Place Order 6.Process 7.Exit
Enter choice: 2

----- MENU -----
1. Burger - Rs 150.00
2. Pizza - Rs 200.00
3. Coffee - Rs 80.00
4. Noodles - Rs 60.00
5. Cholo Bhature - Rs 250.00
6. Cutlet - Rs 120.00
Enter item id: 1
Enter qty: 1
Added Burger x1

1.Menu 2.Add 3.Undo 4.Bill 5.Place Order 6.Process 7.Exit
Enter choice: 2

----- MENU -----
1. Burger - Rs 150.00
2. Pizza - Rs 200.00
3. Coffee - Rs 80.00
4. Noodles - Rs 60.00
5. Cholo Bhature - Rs 250.00
6. Cutlet - Rs 120.00
Enter item id: 2
Enter qty: 2
Added Pizza x2
```

Menu



Burger
Rs. 150
[Add](#)



Pizza
Rs. 200
[Add](#)



Coffee
Rs. 80
[Add](#)



Noodles
Rs. 60
[Add](#)



Cholo Butture
Rs. 250
[Add](#)



Cutlet
Rs. 120
[Add](#)

Current Order

**Burger**
Rs. 150

- 1 +

**Pizza**
Rs. 200

- 2 +

Subtotal : Rs. 550.00
Discount (10%) : Rs. 55.00
Tax (5%) : Rs. 27.50
Total : Rs. 522.50

Payment Method

☒ Cash ☐ E-Wallet ☐ Credit

Undo (Stack)

Clear Cart

Place Order

Undo Operation Output

```
1.Menu 2.Add 3.Undo 4.Bill 5.Place Order 6.Process 7.Exit
Enter choice: 3
Removed Pizza
```

Menu



Burger
Rs. 150
Add



Pizza
Rs. 200
Add



Coffee
Rs. 80
Add



Noodles
Rs. 60
Add



Cholo Butture
Rs. 250
Add



Cutlet
Rs. 120
Add

Current Order



Burger
Rs. 150

-

1

+



Pizza
Rs. 200

-

1

+

Subtotal : Rs. 350.00

Discount (10%) : Rs. 35.00

Tax (5%) : Rs. 17.50

Total : Rs. 332.50

Payment Method

☒ Cash ☐ E-Wallet ☐ Credit

Undo (Stack)

Clear Cart

Place Order

Billing And Processing Order Screen

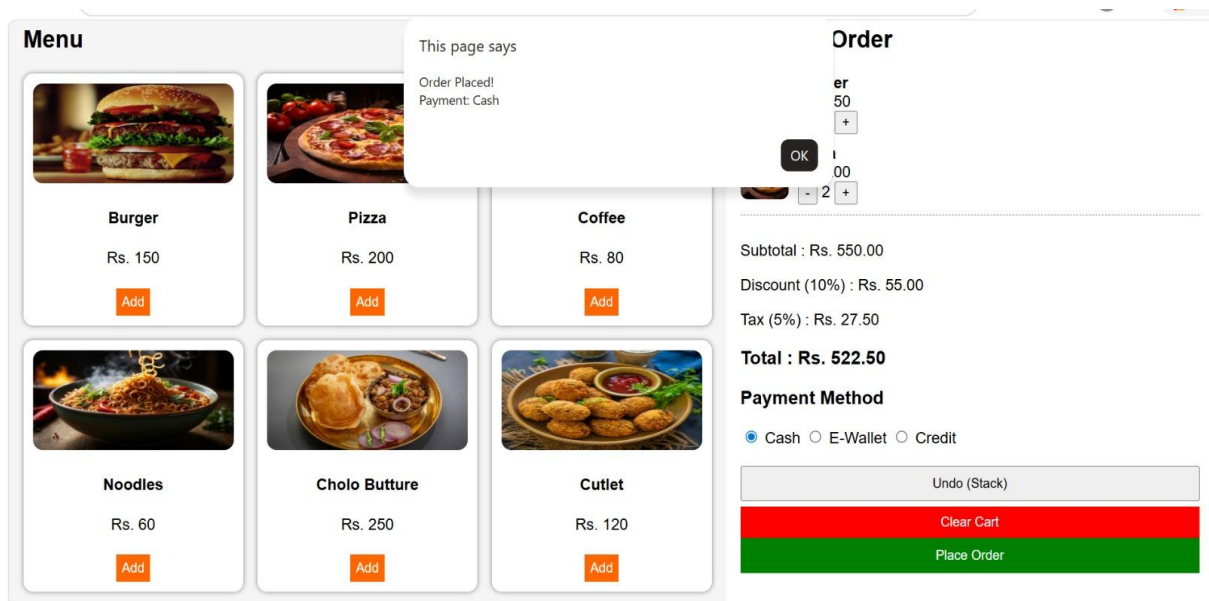
```
1.Menu 2.Add 3.Undo 4.Bill 5.Place Order 6.Process 7.Exit
Enter choice: 4

----- CART -----
Burger x1 = Rs 150.00
Pizza x2 = Rs 400.00

Subtotal: Rs 550.00
Discount (10%): Rs 55.00
Tax (5%): Rs 27.50
TOTAL: Rs 522.50

1.Menu 2.Add 3.Undo 4.Bill 5.Place Order 6.Process 7.Exit
Enter choice: 5
Order placed successfully!

1.Menu 2.Add 3.Undo 4.Bill 5.Place Order 6.Process 7.Exit
Enter choice: 6
Processing Burger
```



4.2 Discussion

The Cafeteria Management and Ordering System is developed as an efficient digital solution to simplify and automate the food ordering and management process in a cafeteria environment, replacing traditional manual methods that are often time-consuming and error-prone. The project begins with requirement analysis, where customer and administrator needs such as menu access, order placement, and billing are identified. Based on these requirements, a systematic system design is created with a user-friendly interface and organized database structure. During development, suitable data structures are implemented to improve system efficiency. Arrays are used to store and display menu items dynamically, while a queue mechanism manages customer orders in a first-come, first-served manner to ensure fair order processing.

A major feature of this project is the automated billing system, which calculates the total amount accurately and reduces manual calculation errors. Essential data such as customer details, orders, and transactions are stored efficiently to ensure proper data handling and retrieval. The system also includes input validation and error-handling techniques to provide reliable performance and smooth user experience. After development, the application is tested under different conditions to verify its functionality and performance. Overall, the project demonstrates how data structures and automation techniques can improve the efficiency, accuracy, and reliability of cafeteria operations while reducing human effort and simplifying management activities.

CHAPTER 5

CONCLUSION

The Cafeteria Management and Ordering System successfully demonstrates how data structures and programming concepts can be used to automate cafeteria operations in an efficient and organized manner. The system simplifies menu management, food ordering, order processing, and billing by replacing manual methods with a digital solution. Arrays are used for storing menu items, queues are used for processing customer orders fairly, and stacks are used for undo operations during ordering. The automated billing system helps calculate the total amount accurately with discount and tax, reducing manual errors and saving time. Overall, the project improves efficiency, reduces workload, and provides a simple and user-friendly solution for cafeteria management. Efficient data handling and proper system design help provide smooth user interaction and better cafeteria management. Overall, the project improves operational efficiency, reduces workload, and provides a user-friendly solution for managing cafeteria activities.

Future Enhancement

The Cafeteria Management and Ordering System can be further improved by adding advanced features and modern technologies. Online payment integration can be implemented to allow customers to make secure digital payments through UPI, cards, or mobile banking applications. Real-time order tracking and notification systems can also be added to inform customers about order status and estimated delivery time. The system can be enhanced with customer login and account management features to maintain order history and personalized preferences.

REFERENCES

- [1] **“Python Crash Course”** Eric Matthes, **2nd Edition**, 2019, No Starch Press
- [2] **“Data Structures and Algorithms Made Easy”** Narasimha Karumanchi, **2016 Edition**, CareerMonk Publications
- [3] **“Head First Python”** Paul Barry, **2nd Edition**, 2016, O’Reilly Media
- [4] **“Learning Python”** Mark Lutz, **5th Edition**, 2013, O’Reilly Media
- [5] **“Database System Concepts”** Abraham Silberschatz, Henry F. Korth, S. Sudarshan, **6th Edition**, 2010, McGraw-Hill
- [6] **GeeksforGeeks**
<https://www.geeksforgeeks.org/python-programming-language>
- [7] **W3Schools**
<https://www.w3schools.com/python>
- [8] **TutorialsPoint**
<https://www.tutorialspoint.com/python/index.htm>
- [9] **Programiz**
<https://www.programiz.com/python-programming>
- [10] **Real Python**
<https://realpython.com>

APPENDIX

```
#include <stdlib.h>
#include <string.h>
#define MAX 10
// ----- MENU -----
struct Menu {
    int id;
    char name[50];
    float price;
};
struct Menu menu[] = {
    {1, "Burger", 150},
    {2, "Pizza", 200},
    {3, "Coffee", 80},
    {4, "Noodles", 120},
    {5, "Cholo Bhature", 140},
    {6, "Cutlet", 100}
};
int menuSize = 6;
// ----- CART -----
struct CartItem {
    int id;
    int qty;
    float total;
};
struct CartItem cart[MAX];
int cartSize = 0;
// ----- STACK (UNDO) -----
```

```

struct CartItem stack[MAX];
int top = -1;
// ----- QUEUE (ORDER PROCESS) -----
struct CartItem queue[MAX];
int front = -1, rear = -1;
// ----- FUNCTIONS -----
float getPrice(int id) {
    for (int i = 0; i < menuSize; i++) {
        if (menu[i].id == id)
            return menu[i].price;
    }
    return 0;
}
char* getName(int id) {
    for (int i = 0; i < menuSize; i++) {
        if (menu[i].id == id)
            return menu[i].name;
    }
    return "";
}
// ----- DISPLAY MENU -----
void showMenu() {
    printf("\n----- MENU ----- \n");
    for (int i = 0; i < menuSize; i++) {
        printf("%d. %s - Rs %.2f\n", menu[i].id, menu[i].name, menu[i].price);
    }
}
// ----- ADD TO CART -----

```

```

void addItem(int id, int qty) {
    float price = getPrice(id);
    if (price == 0) {
        printf("Invalid item!\n");
        return;
    }
    struct CartItem item;
    item.id = id;
    item.qty = qty;
    item.total = price * qty;
    cart[cartSize++] = item;
    // push to stack
    stack[++top] = item;
    printf("Added %s x%d\n", getName(id), qty);
}
// ----- UNDO -----
void undo() {
    if (top == -1) {
        printf("Nothing to undo!\n");
        return;
    }
    printf("Removed %s\n", getName(stack[top].id));
    cartSize--;
    top--;
}
// ----- BILL -----
void showBill() {
    float subtotal = 0

```



```

printf("\n----- CART ----- \n");
for (int i = 0; i < cartSize; i++) {
    printf("%s x%d = Rs %.2f\n",
        getName(cart[i].id),
        cart[i].qty,
        cart[i].total);
    subtotal += cart[i].total;
}
float discount = subtotal * 0.10;
float tax = subtotal * 0.05;
float total = subtotal - discount + tax;

printf("\nSubtotal: Rs %.2f\n", subtotal);
printf("Discount (10%%): Rs %.2f\n", discount);
printf("Tax (5%%): Rs %.2f\n", tax);
printf("TOTAL: Rs %.2f\n", total);
}
// ----- QUEUE -----
void placeOrder() {
    if (cartSize == 0) {
        printf("Cart empty!\n");
        return;
    }
    for (int i = 0; i < cartSize; i++) {
        if (rear == MAX - 1) {
            printf("Queue full!\n");
            return;
        }
    }
}

```

```

    if (front == -1) front = 0
        queue[++rear] = cart[i];
    }
    printf("Order placed successfully!\n");
// clear cart
    cartSize = 0;
    top = -1;
}
// ----- PROCESS ORDER -----
void processOrder() {
    if (front == -1 || front > rear) {
        printf("No orders!\n");
        return;
    }
    printf("Processing %s\n", getName(queue[front].id));
    front++;
}
// ----- MAIN -----
int main() {
    int choice, id, qty;
    while (1) {
        printf("\n1.Menu 2.Add 3.Undo 4.Bill 5.Place Order 6.Process
7.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                showMenu();

```

```

        break;
    case 2:
        showMenu();
        printf("Enter item id: ");
        scanf("%d", &id);
        printf("Enter qty: ");
        scanf("%d", &qty);
        addItem(id, qty);
        break;
    case 3:
        undo();
        break;
    case 4:
        showBill();
        break;
    case 5:
        placeOrder();
        break;
    case 6:
        processOrder();
        break;
    case 7:
        exit(0);
    default:
        printf("Invalid!\n");
    }
}
}

```